

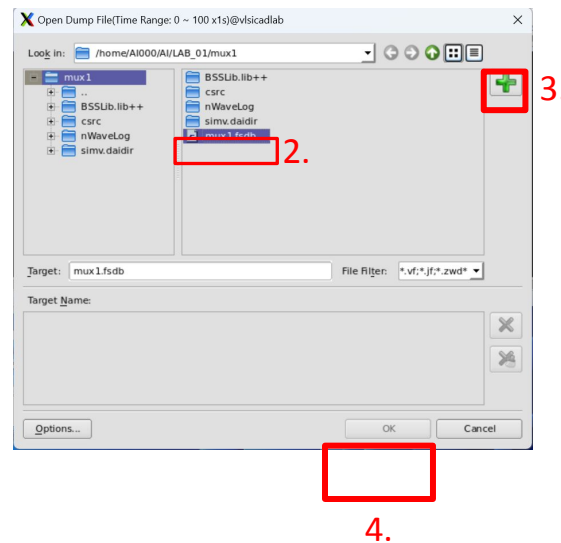
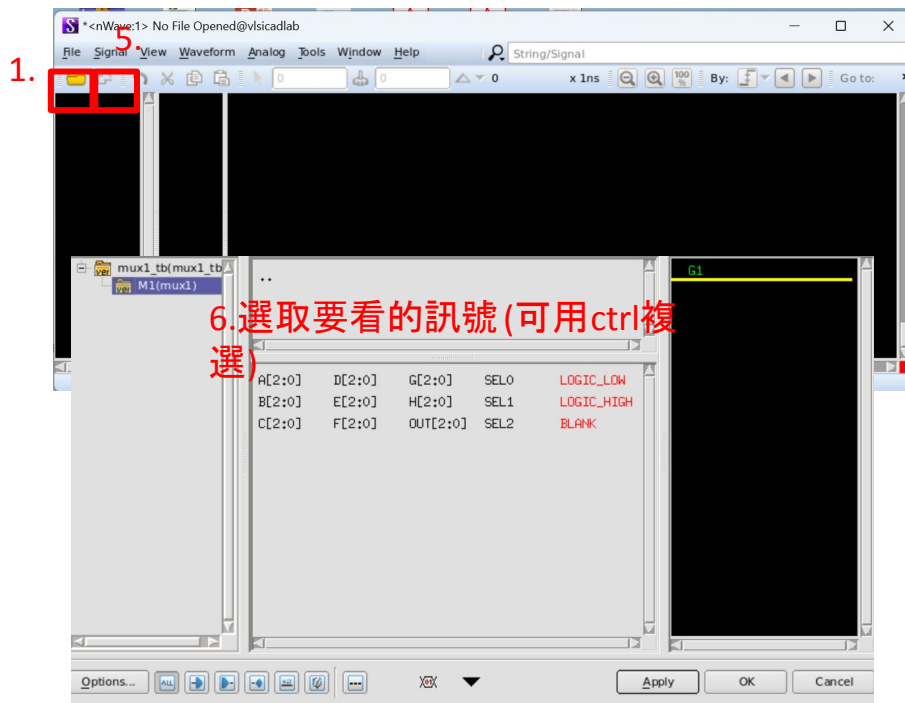
工作站指令介紹

`vcs -R (測試檔) (程式檔) -full64 -debug_access+all +v2k | tee (模擬結果儲存檔名)`

`vcs -R test_tb.v test.v -full64 -debug_access+all +v2k | tee test.log`

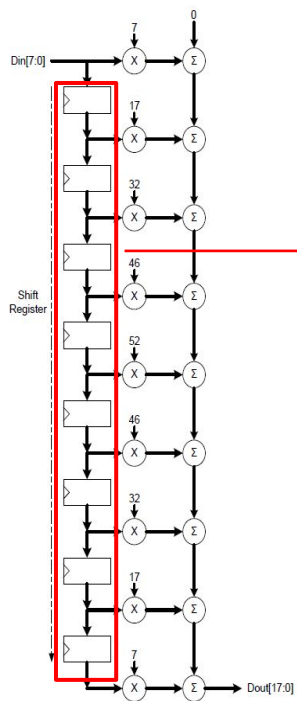
`nWave &` (*看波型)

nWave 使用



7. 選好後按Apply再按
ok

FIR



提示1.For 迴圈

```
for ( i = (起始值); (迴圈執行條件); i = i + 1)
    執行動作
end
```

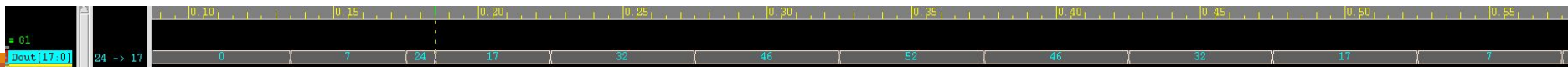
提示2.用多個暫存器(REG)存值

*for 迴圈使用限制:

迴圈限制必須為常數(需要先固定)

多個reg在rst時歸零, 需要以另一個 for迴圈做出歸零

效果



管線化(Pipeline)

透過把計算過程切割成多個小部分以解決Setup Time的問題。

* 但這會使計算時長倍數增加

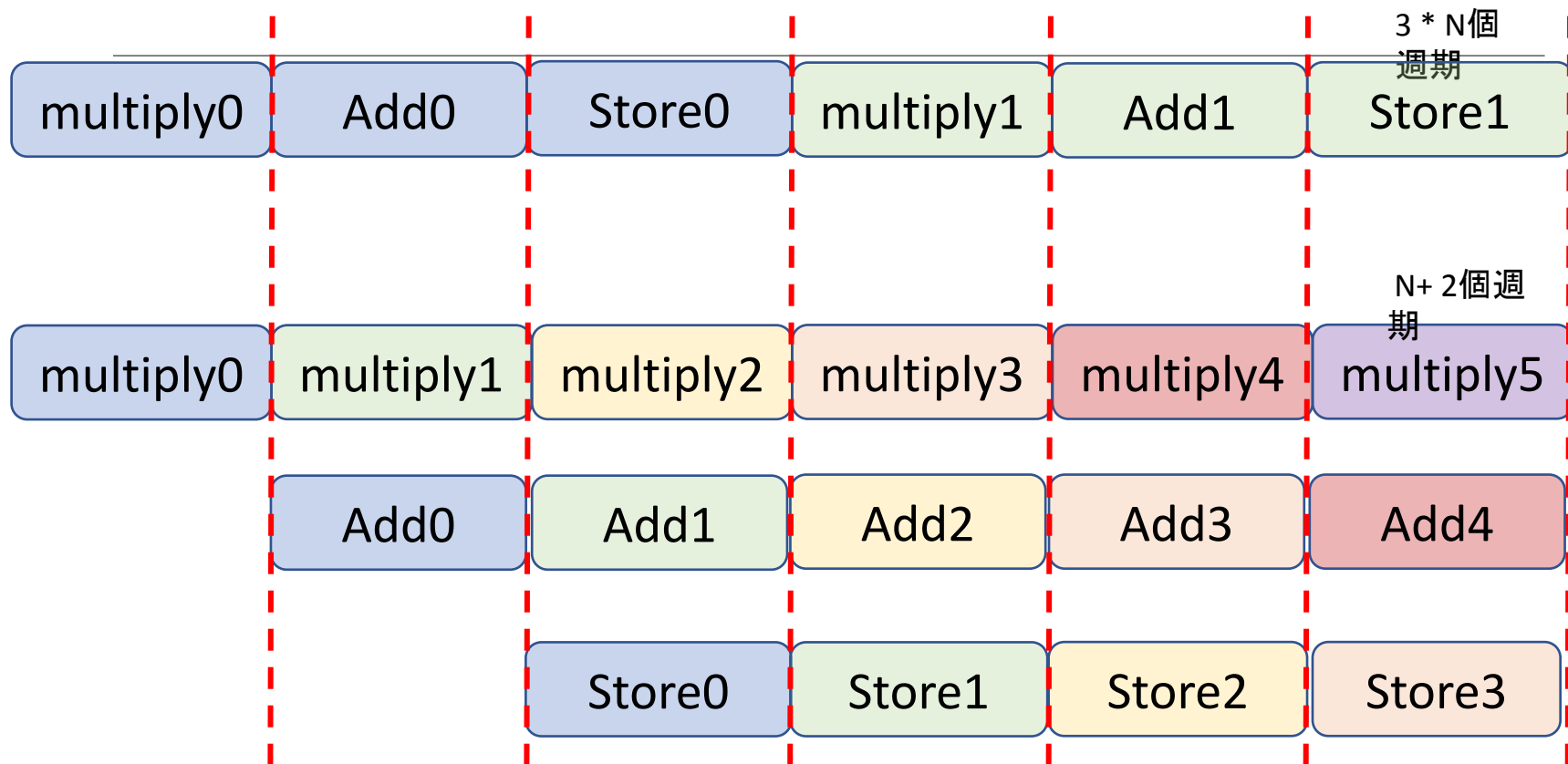


N個週
期



3 * N個週
期

管線化(Pipeline)



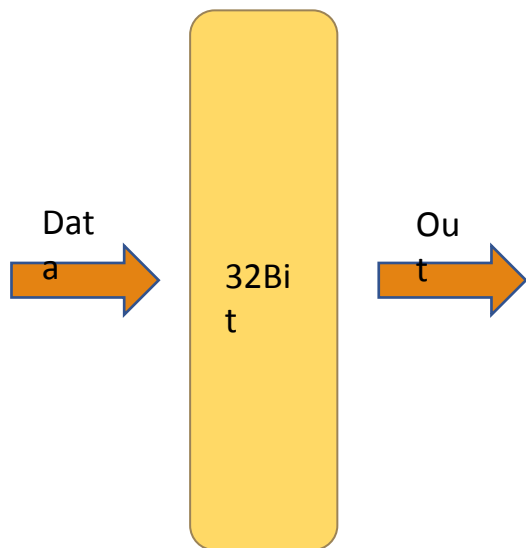
LAB1 32bits優先進位編碼器

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

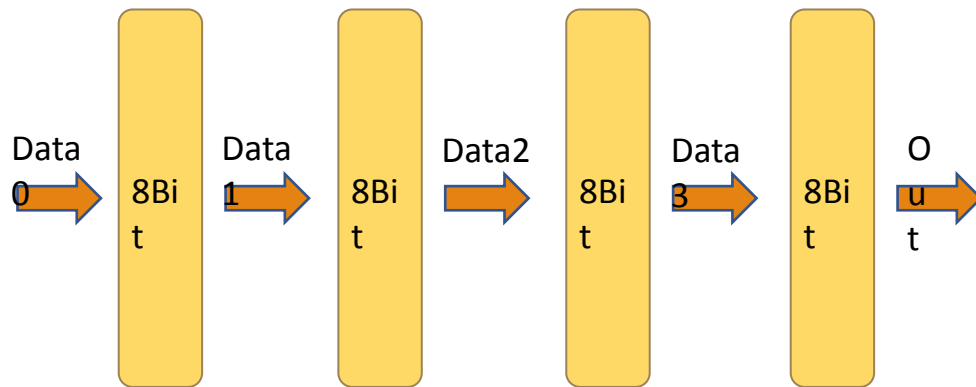
是否找到
值

LAB1 32bits優先進位編碼器

原架構：



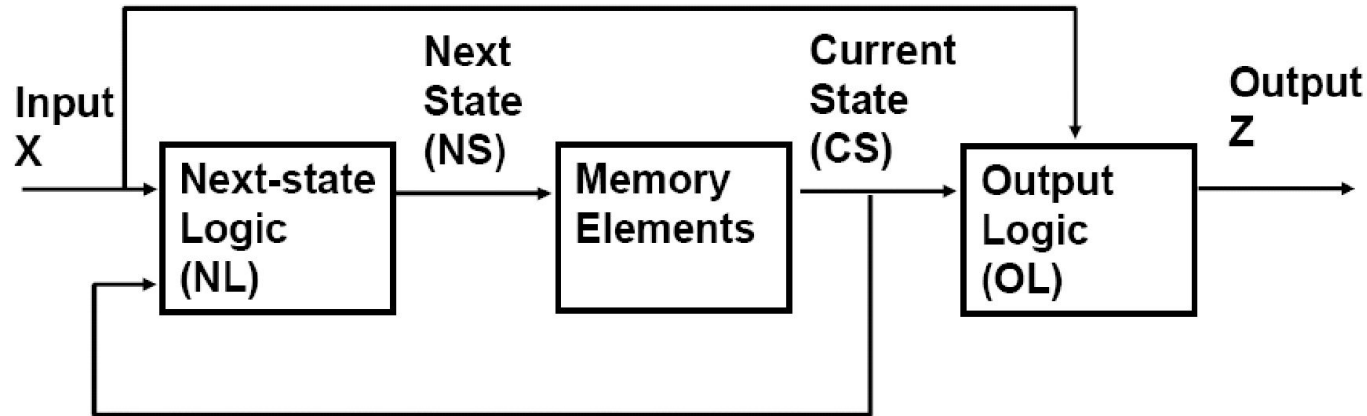
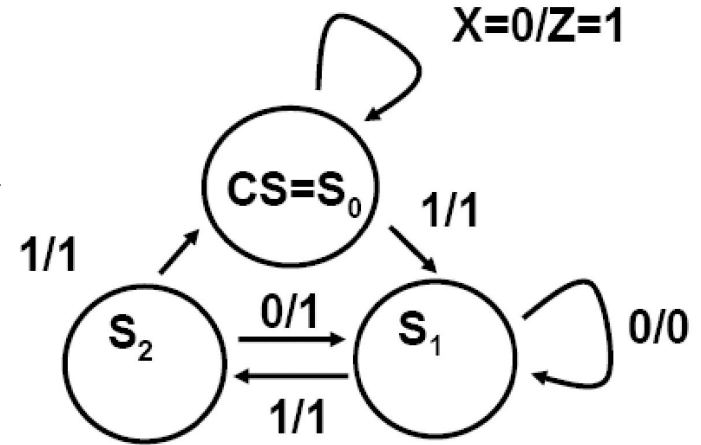
Pipeline後：



Finite-State Machine

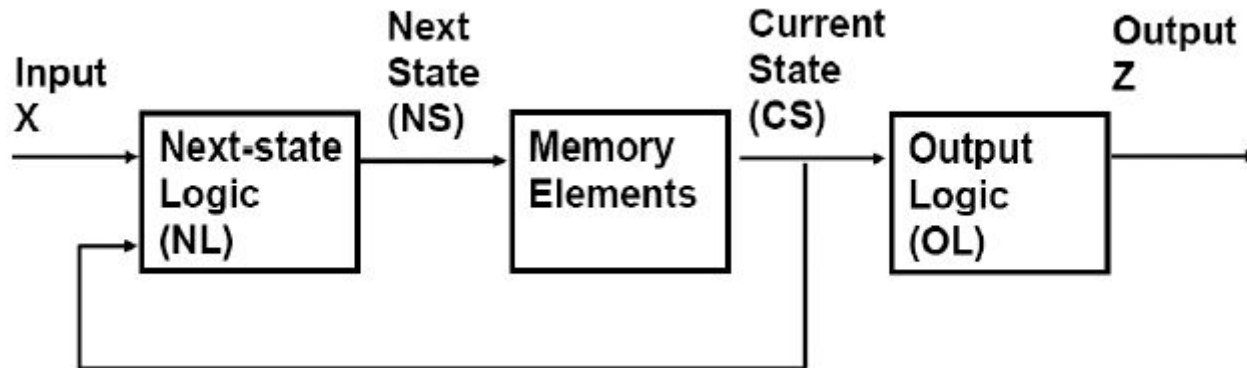
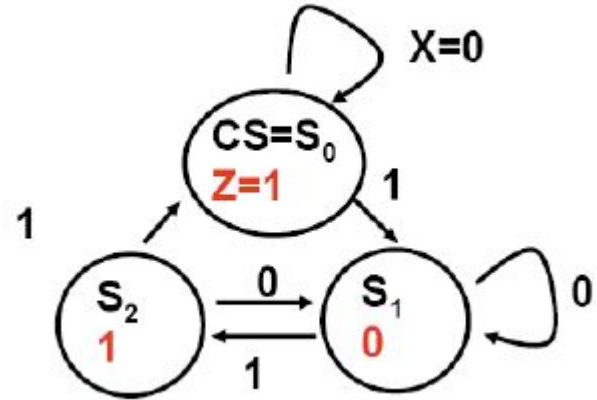
Mealy Machine

- Output is a function of both current state & input



Moore Machine

- Output is a function of only current state



Behavior Modeling of FSM

- Combinational Part

- Next-state logic (NL)
- Output logic (OL)

- Sequential Part

- Current state (CS) stored in flip-flops

- 3 Coding Style

- Separate CS, OL and NL
- Combine NL+ OL, separate CS
- Combine CS + NL, separate OL

Coding Style 1: Separate CS, OL and NL

- CS

```
always @ (posedge clk)
    current_state <= next_state;
```

- NL

```
always @ (current_state or In)
    case (current_state)
        S0: case (In)
            In0: next_state = S1;
            In1: next_state = S0;
            . . .
        endcase //In
        S1: . . .
        S2: . . .
    endcase //current_state
```

- OL

```
// if Moore
always @ (current_state)
    Z = output_value;
```

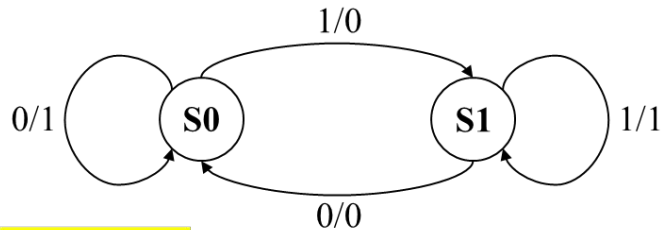
```
// if Mealy
always @ (current_state or In)
    Z = output_value;
```

Coding Style 1: Separate CS, OL and NL

```
module FSM(in, out, clk, reset);  
input  in, clk, reset;  
output out;  
reg    out, current_state, next_state;  
  
parameter  state0 = 1'b0, statel = 1'b1;  
  
always@(posedge clk or posedge reset)  
begin    //CS  
    if(reset) current_state <= state0;  
    else current_state <= next_state;  
end
```

```
always@(current_state or in)  
begin    //NS  
    case(current_state)  
        state0 : next_state = (in == 1'b0) ? state0 : statel;  
        statel : next_state = (in == 1'b0) ? state0 : statel;  
    endcase  
end
```

```
always@(current_state or in)  
begin    //OL  
    case(current_state)  
        state0 : out = (in == 1'b0) ? 1'b1 : 1'b0;  
        statel : out = (in == 1'b0) ? 1'b0 : 1'b1;  
    endcase  
end  
  
endmodule
```



Input/Output

Coding Style 2: Combine NL+ OL, Separate CS

- CS

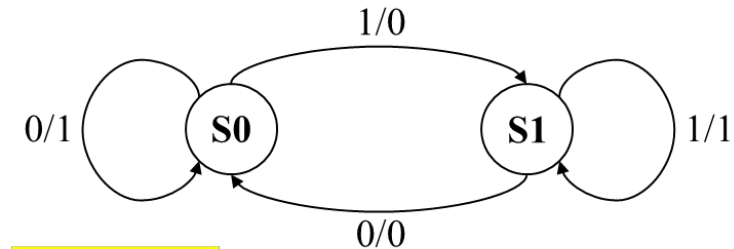
```
always @ (posedge clk)
    current_state <= next_state;
```

- NL+OL

```
always @ (current_state or In)
    case (current_state)
        S0: begin
            case (In)
                In0: begin
                    next_state = S1;
                    Z =values; // Mealy
                end
                In1: . . .
            endcase // In
            Z =values; // Moore
        end //S0
        S1: . . .
    endcase // current_state
```

Coding Style 2: Combine NL+ OL, Separate CS

```
module FSM(in, out, clk, reset);  
input    in, clk, reset;  
output   out;  
reg      out, current_state, next_state;  
  
parameter state0 = 1'b0, state1 = 1'b1;  
  
always@(posedge clk or posedge reset)  
begin    //CS  
    if(reset) current_state <= state0;  
    else current_state <= next_state;  
end
```



Input/Output

```
always@(current_state or in)  
begin    //NS and OL  
    case(current_state)  
        state0 : begin  
            case(in)  
                1'b0 : begin next_state = state0; out = 1'b1; end  
                1'b1 : begin next_state = state1; out = 1'b0; end  
            endcase  
        end  
        state1 : begin  
            case(in)  
                1'b0 : begin next_state = state0; out = 1'b0; end  
                1'b1 : begin next_state = state1; out = 1'b1; end  
            endcase  
        end  
    endcase  
end  
  
endmodule
```

Coding Style 3: Combine CS+NL, Separate OL

- CS+NL

- OL

```
always @ (posedge clk)
begin
    case (state)
        S0: case (In)
            In0: state<= S1;
            In1: state<= S0;
            . . .
        endcase //In
        S1: . . .
    endcase //state
end
```

```
// if Moore
always @ (state)
    Z = output_value;
```

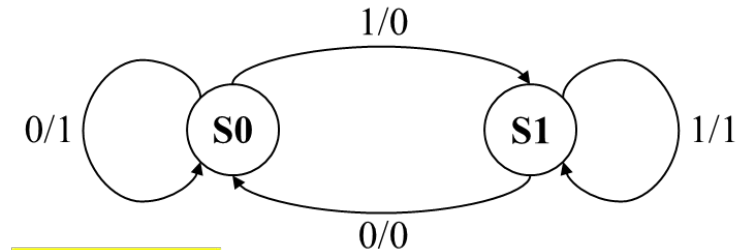
```
// if Mealy
always @ (state or In)
    Z = output_value;
```

Coding Style 3: Combine CS+NL, Separate OL

```
module FSM(in, out, clk, reset);  
input  in, clk, reset;  
output out;  
reg    out, current_state;  
  
parameter state0 = 1'b0, state1 = 1'b1;
```

```
always@(posedge clk or posedge reset)  
begin    //CS and NS  
    if(reset) current_state <= state0;  
    else begin  
        case(current_state)  
            state0 : current_state <= (in == 1'b0) ? state0 : state1;  
            state1 : current_state <= (in == 1'b0) ? state0 : state1;  
        endcase  
    end  
end
```

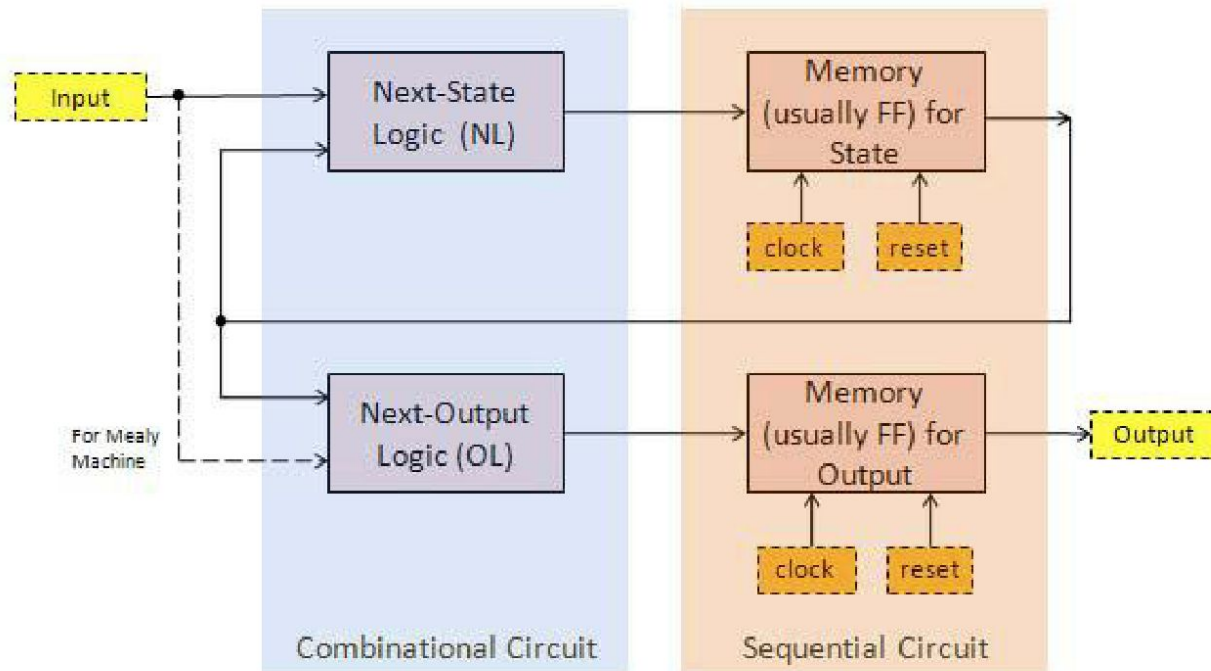
```
always@(current_state or in)  
begin    //OL  
    case(current_state)  
        state0 : out = (in == 1'b0) ? 1'b1 : 1'b0;  
        state1 : out = (in == 1'b0) ? 1'b0 : 1'b1;  
    endcase  
end  
  
endmodule
```



Input/Output

Architecture of FSM

Build combinational and sequential parts separately!



FSM Design Notice

- Partition FSM and non-FSM logic
- Partition combinational part and sequential part
 - Coding style 1, 2 are preferred
 - For beginner, Do not use Coding style 3
- Use `parameter` to define names of the state vector
- Assign a default (reset) state